

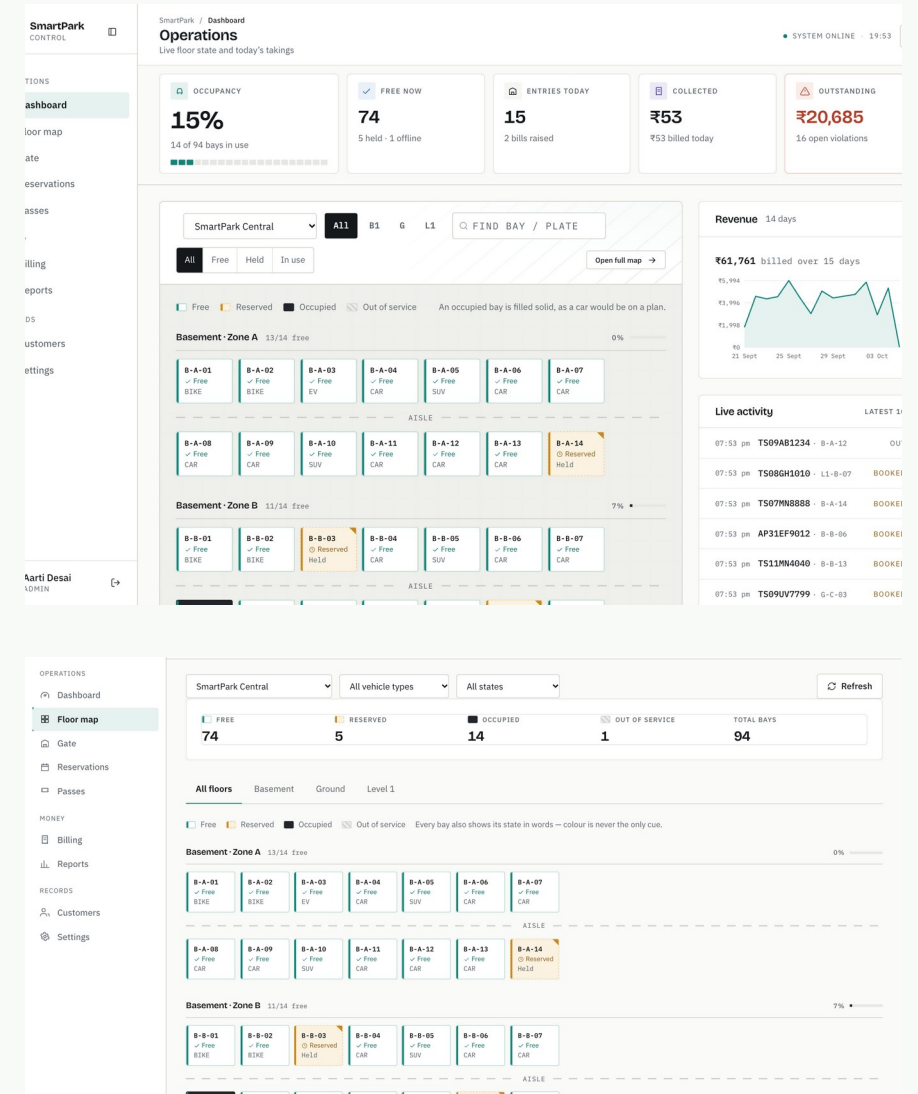


DBMS Final Presentation

Design and Implementation of a Database Management System for Smart Parking Lot Allocation and Billing System

Project 20, Group 12
Lakshay Sethia, Sidharth Sood, Shardul Yadav, Naman Singla
Repository commit: 16c5f23

DBMS final presentation, verified against the current GitHub repository



Problem Description

Multi-level parking needs one trusted record of bays, vehicles, reservations, entries, exits and payments. Manual ticketing fails when slot availability changes faster than staff can coordinate.

Failures to remove

Slot conflicts
Two attendants can assign the same bay

Revenue leakage
Charges can vary by person and unpaid exits may be missed

Core objective

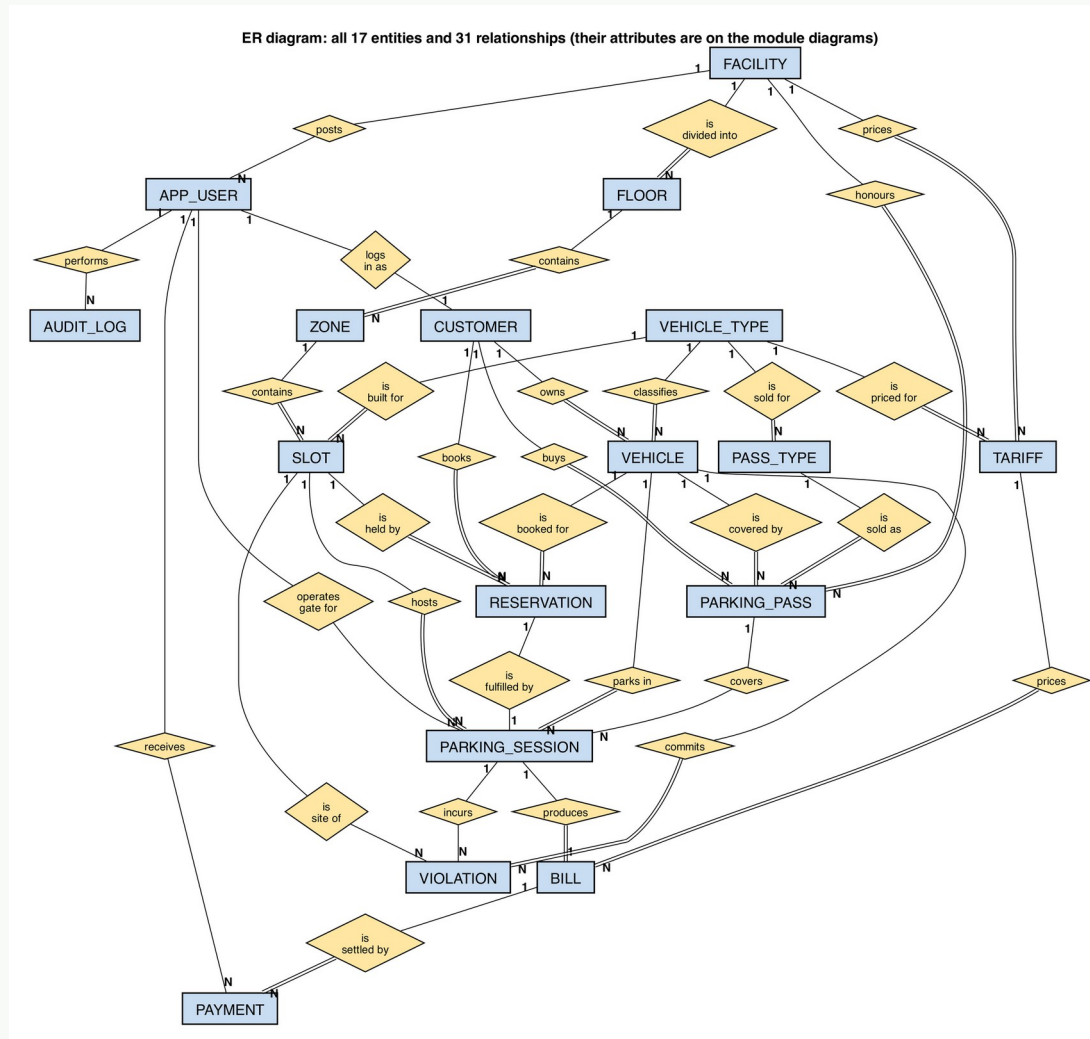
Make invalid parking states hard to store by using keys, constraints, triggers, transactions and role-based access in the database.

Users and scope

Administrators
Gate operators
Customers

No mobile app, no hardware sensor integration, no payment gateway build-out

ER Diagram and Relationships



Entities

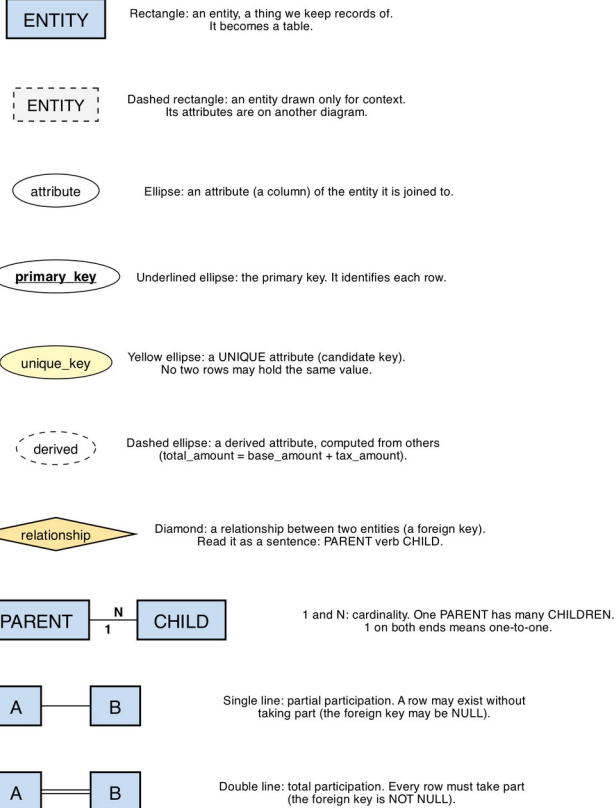
17 tables model the domain:
facility, floor, zone, slot, vehicle type,
customer, vehicle, reservation,
parking session, pass, tariff, bill,
payment, violation, app user, pass
type, audit log.

Cardinality to defend

Facility to floor: 1 to many
Vehicle to session: 1 to many over time, 1
active at a time
Slot to reservation: 1 to many, non-
overlapping while live
Session to bill: 1 to 0 or 1

ER Notation Key (Chen)

How to read the ER diagrams (Chen notation)



Foreign-key columns are not drawn as attributes; the relationship diamond is the foreign key.
Not used here: weak entities (double rectangle) and multivalued attributes (double ellipse). Every entity has its own primary key, and repeating groups such as payments were moved into their own entity during normalization.

How to read every ER diagram

Rectangle: an entity (a table). Dashed rectangle: context entity shown for reference.

Diamond: a relationship, labelled with a verb; 1 / N on each line is the cardinality.

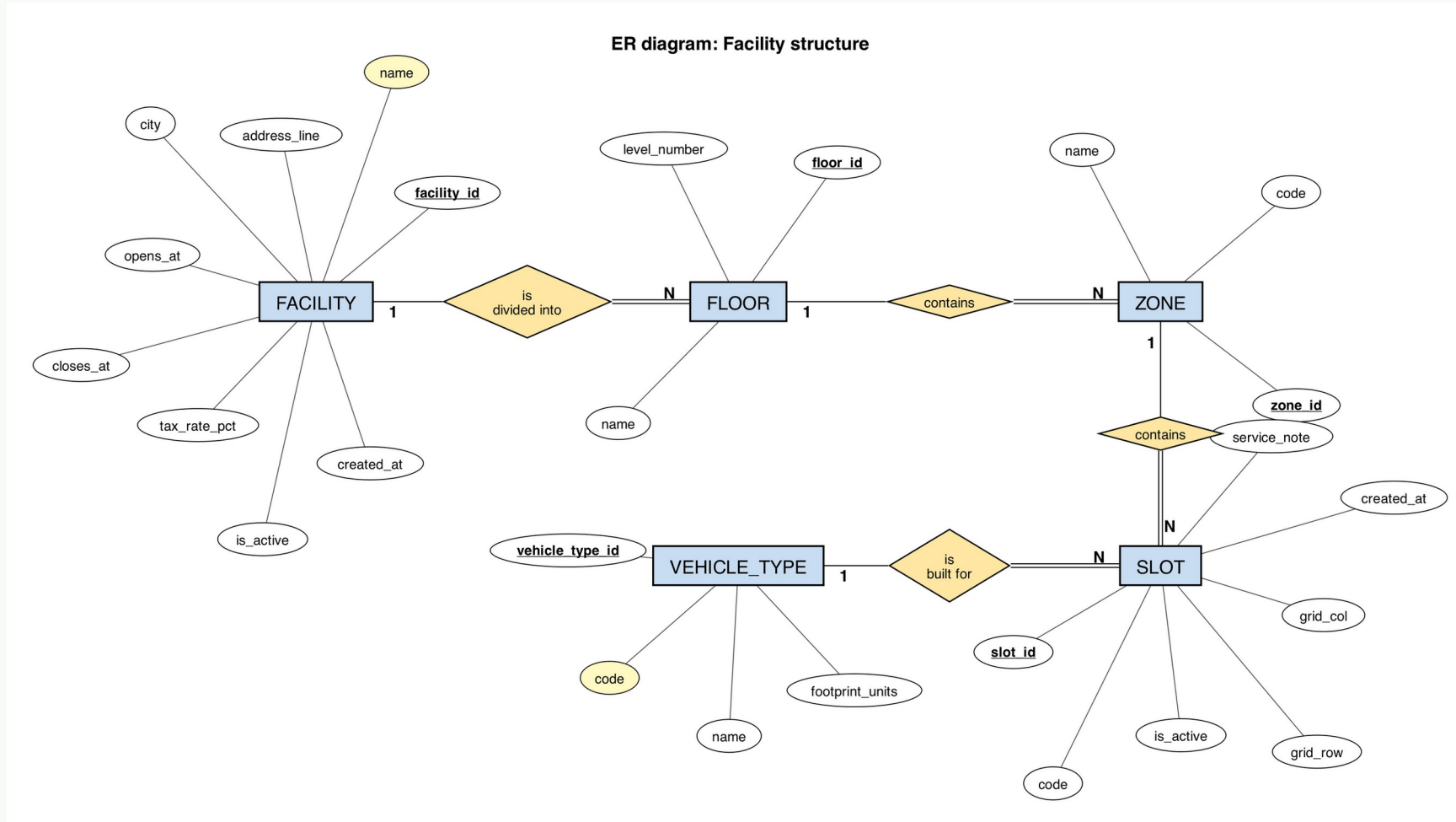
Single line: partial participation (nullable foreign key). Double line: total participation (NOT NULL foreign key).

Underlined ellipse: primary key. Yellow ellipse: UNIQUE. Dashed ellipse: derived attribute.

Foreign-key columns are not drawn as attributes; they are the relationship lines.

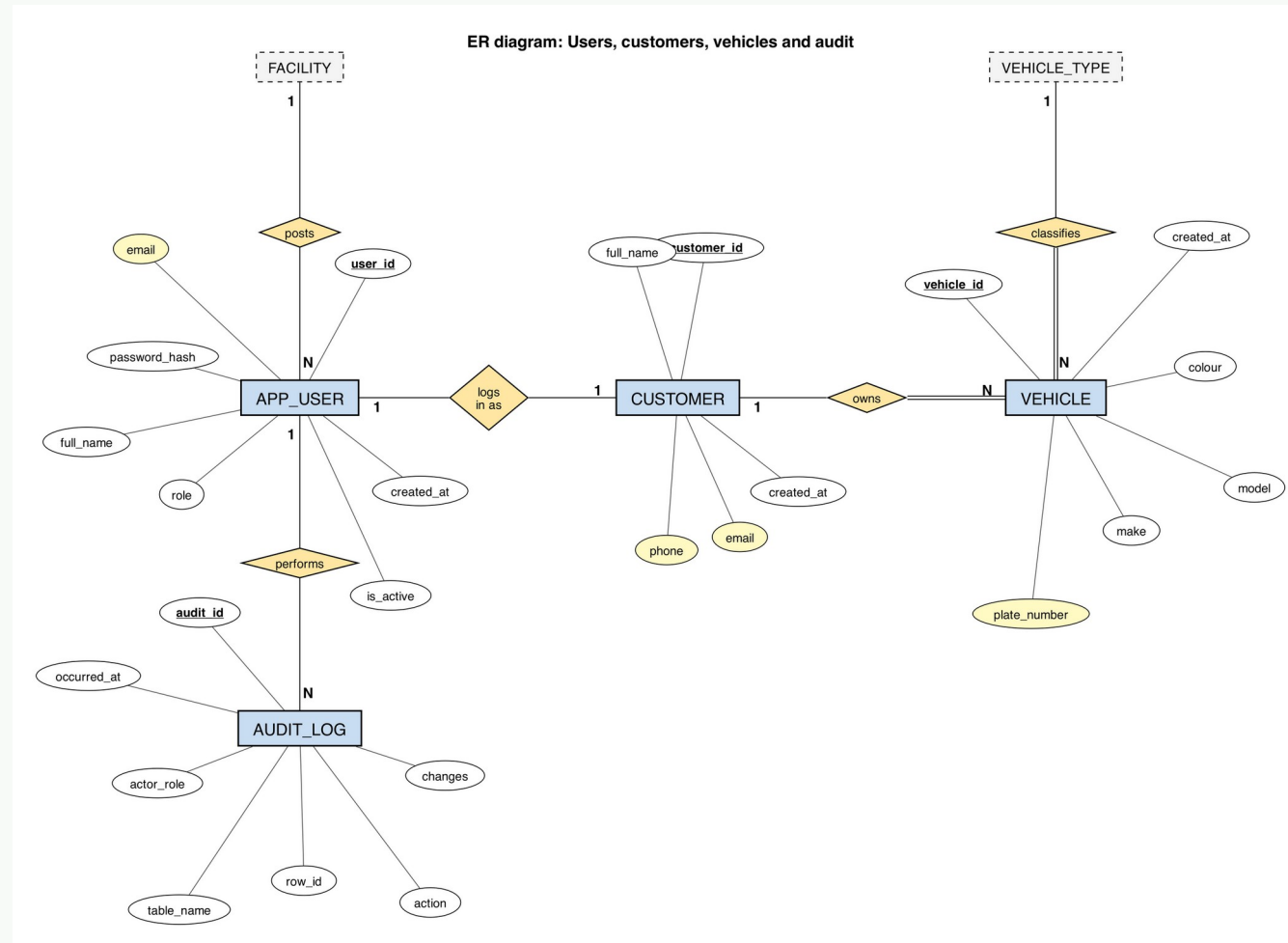
All cardinalities and participation are read from the live database constraints, so the diagram cannot drift from the schema.

ER Detail: Facility structure



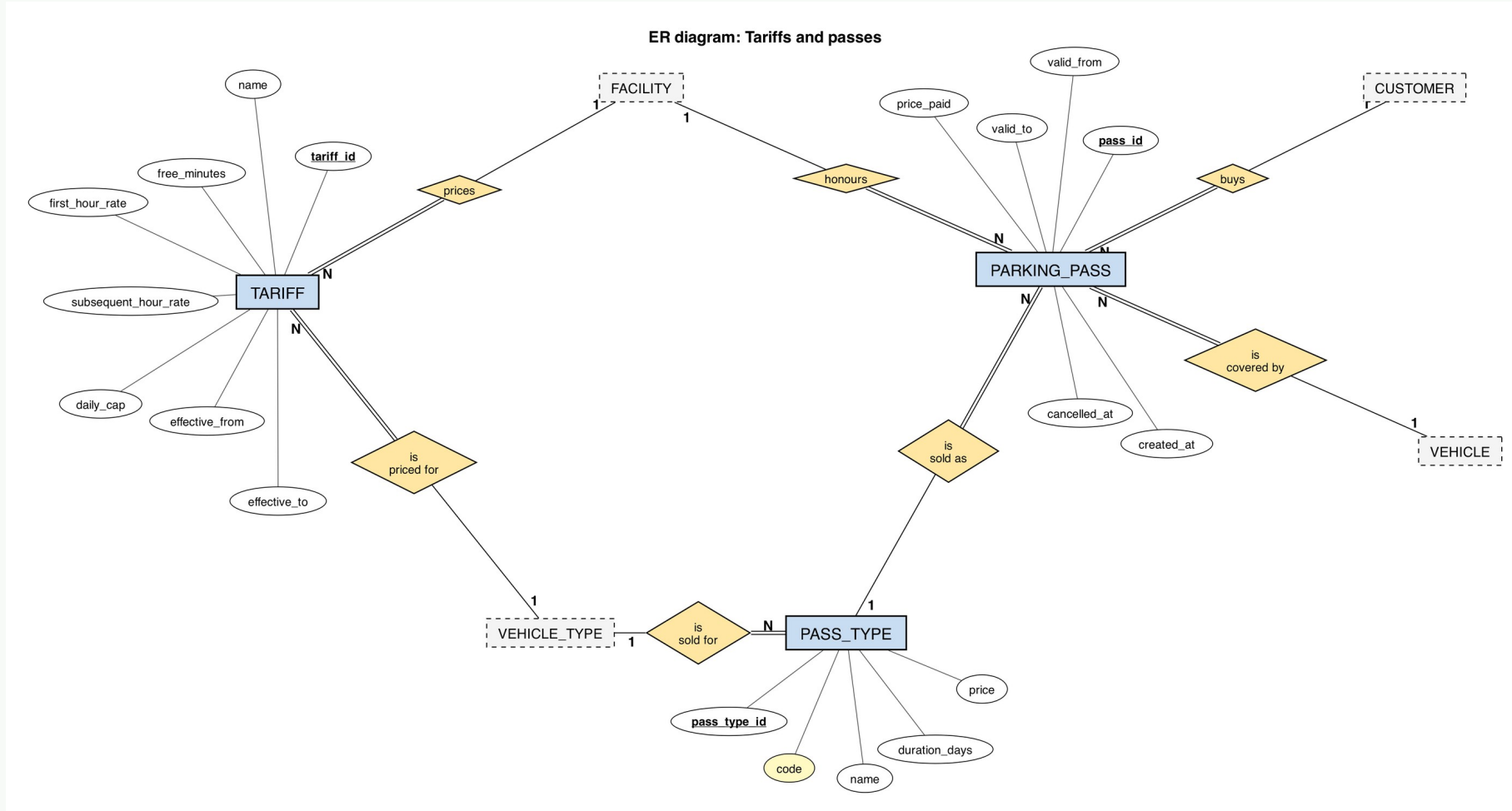
Facility to floor to zone to slot; slot is built for one vehicle type.

ER Detail: Users, customers, vehicles and audit



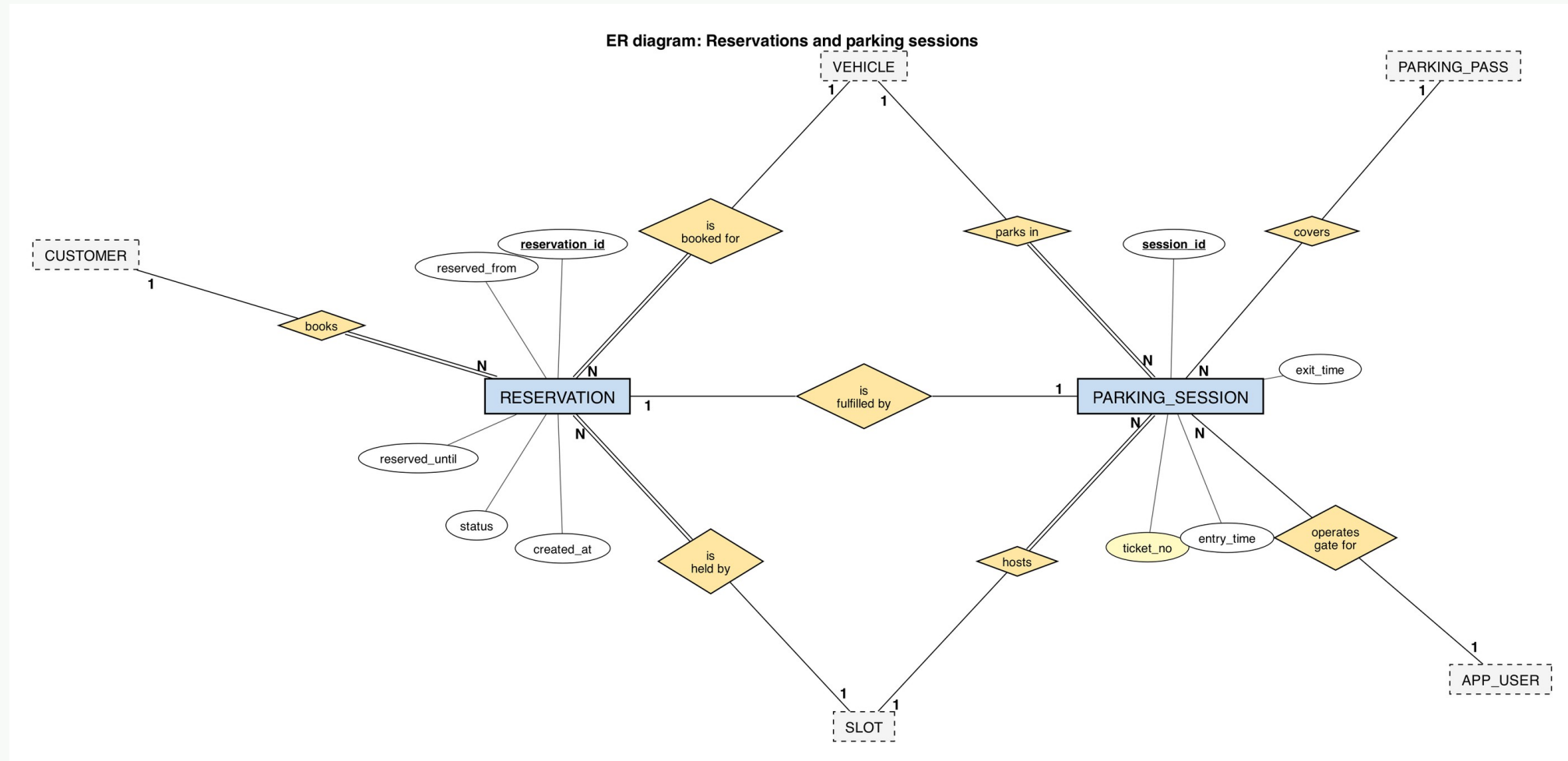
Login identity is separate from the customer record; audit log records changes.

ER Detail: Tariffs and passes



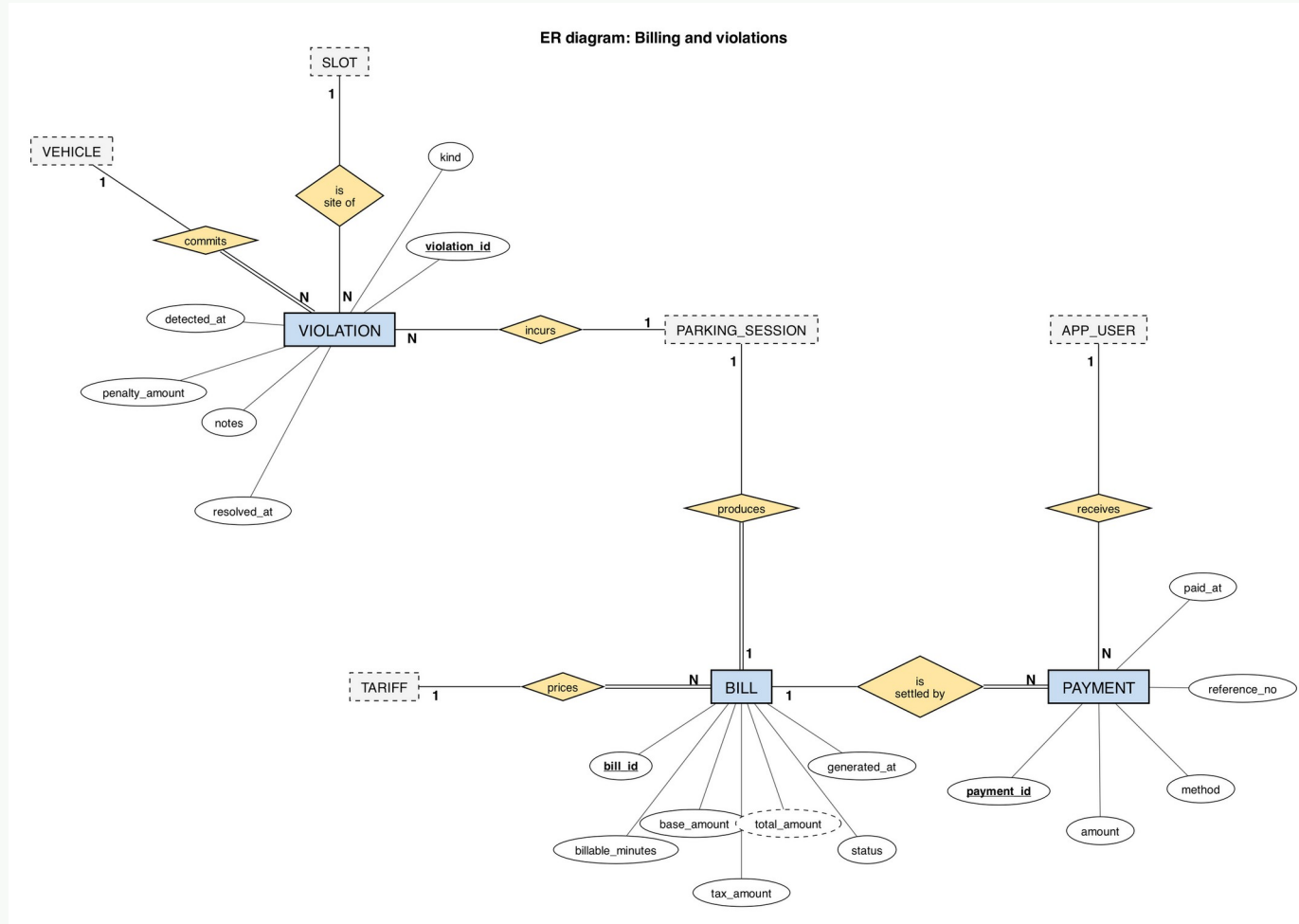
Tariffs are priced per vehicle type; passes bind a customer, vehicle and pass type.

ER Detail: Reservations and parking sessions



Non-overlapping reservations per slot; one active session per slot and vehicle.

ER Detail: Billing and violations



One bill per session, priced by a tariff; payments settle bills.

Relational Schema and 3NF

Schema group	Main tables	Design point
Physical structure	facility, floor, zone, slot	Each bay resolves to one facility through containment
People and vehicles	app_user, customer, vehicle, vehicle_type	Login identity stays separate from customer record
Operations	reservation, parking_session, parking_pass	Open state derives from time fields and exits
Money	tariff, bill, payment, pass_type	Bills keep the tariff that priced the stay
Control	violation, audit_log	Violations and changes remain queryable

Normalization evidence

1NF
Payments and tariff versions move out of the flat ticket record

2NF
Facility, floor, zone and slot split the composite bay key

3NF
Customer, vehicle type and derived status facts are separated

Deliberate absence: no slot.status column. Slot state is derived from open sessions and live reservations, so the database cannot disagree with itself.

Database Design and Constraints

Verified item	Fresh checkout result
Base tables	17
Report views	8
User-defined functions	16
Triggers	14
Foreign keys	36
Indexes	59

Rules enforced in the database

One active session per slot and per vehicle: partial unique indexes
Slot type must match vehicle type: composite foreign keys
No overlapping reservations, passes or tariffs: GiST exclusion constraints
Exit follows entry: CHECK constraint
Charges come from tariff function, then a trigger overwrites client amounts
Payments cannot exceed bill balance

The professor asks for database and table creation with keys, foreign keys and constraints. This project uses PostgreSQL 17, so the live commands are PostgreSQL SQL rather than MySQL-only syntax.

Reservations

Passes

MONEY

Billing

Reports

RECORDS

Customers

Settings

Written by the `trg_audit` trigger on every insert, update and delete. No application role can edit or delete it.

Search person, table or change

5 records

Export CSV

WHEN	WHO	ACTION	RECORD	CHANGE
05 Oct, 07:54 pm	Aarti Desai Admin	Insert	Bill #1610	Created with 9 fields
05 Oct, 07:54 pm	Aarti Desai Admin	Update	Parking Session #1626	exit_time: - - 05 Oct, 07:54 pm, exit_operat...
05 Oct, 07:54 pm	Aarti Desai Admin	Insert	Parking Session #1629	Created with 11 fields
05 Oct, 07:53 pm	Aarti Desai	Insert	Bill #1609	Created with 9 fields

Database and Query Evidence

DDL and DML files

db/migrations/001 to 019
schema, constraints, functions, views, RLS and seed data

docs/database/queries.sql
18 runnable query examples covering joins, aggregate functions, subqueries, views, windows and JSONB

Fresh query output

SmartPark Central, Hatchback / Sedan, 52 bays, 1 out of service
SmartPark Riverside, Hatchback / Sedan, 22 bays, 0 out of service
End-to-end run found 41 free car bays before demo inserts

Best live SQL to demonstrate

1. Show migrations for CREATE TABLE and constraints
2. Run docs/database/queries.sql for joins and aggregates
3. Run constraint test 8 to prove client-entered billing amount gets corrected
4. Run one report view, such as v_revenue_daily or v_free_slots

Constraint test 8 result

client claimed: base 1.00, tax 0.00, total 1.00
database stored: base 80.00, tax 14.40, total 94.40

Transactions and Security

Gate entry as one transaction

fn_gate_entry resolves vehicle, honours a live reservation if present, locks a candidate slot, attaches a valid pass, inserts the session and marks the reservation fulfilled.

Locking detail

```
SELECT ... FOR UPDATE SKIP LOCKED
```

This prevents two operators from receiving the same free bay during concurrent arrivals.

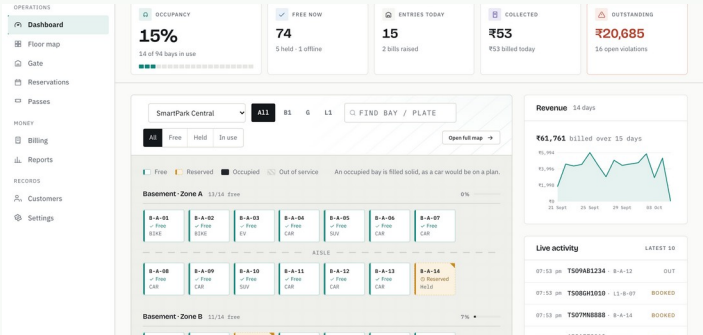
Role-level isolation

FastAPI sets the caller identity inside each transaction:
SET LOCAL app.current_user_id
SET LOCAL ROLE parking_operator or
parking_customer

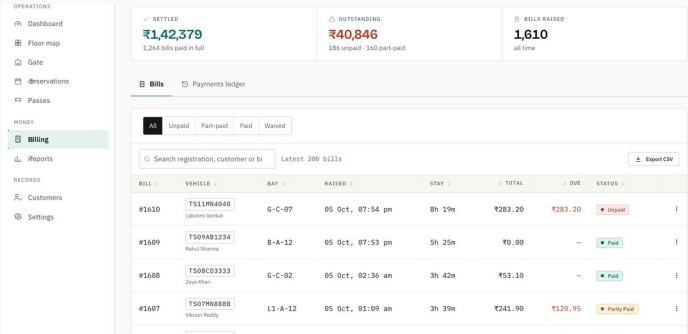
Fresh checks

RLS tests completed
Customer sees 3 vehicles while admin sees 47
Operator cannot change tariffs
Only admins can read audit_log

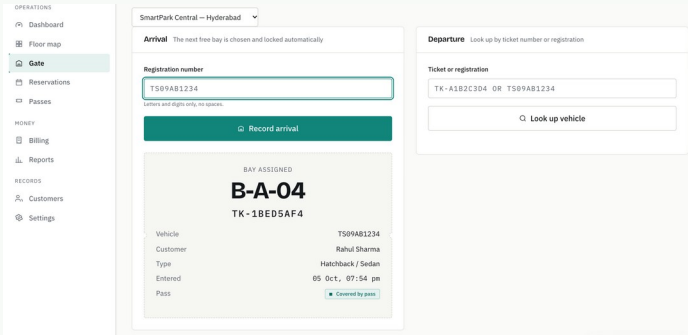
User Interface Demo



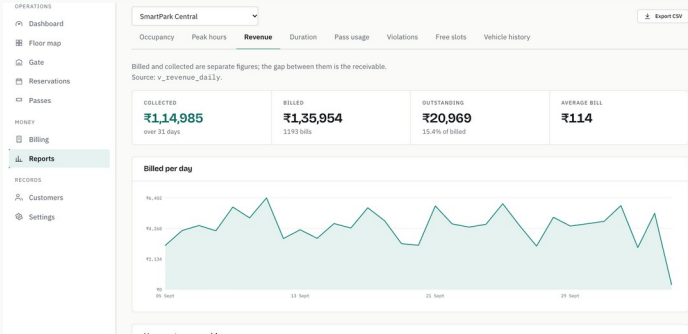
Dashboard



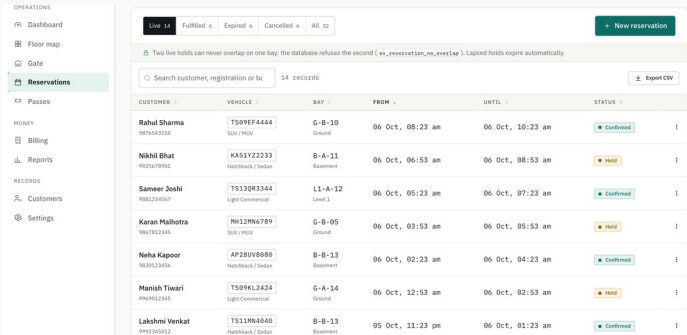
Billing



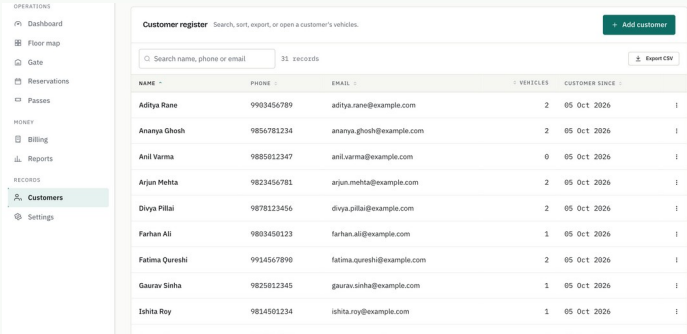
Gate entry



Revenue report



Reservations



Customers

Verification Results

Check	Fresh result from this checkout
Database setup	PostgreSQL 17.11, 19 migrations, 134 bays, 1,541 sessions, 1,521 bills
Constraint tests	Completed, deliberate violations refused or corrected by database
Lifecycle tests	5 lifecycle groups passed, rolled back after execution
RLS tests	Completed, audit trail restricted, facility and customer scoping held
Import check	14 local modules, 24 files scanned, every named import resolves
End-to-end smoke	All checks passed, sign-in to reports through real API
API pytest	Not run here, pytest is not installed by the standard setup venv

Questions

Likely viva questions

Why PostgreSQL instead of MySQL?
Which constraints stop double booking?
How does 3NF apply to the schema?
Why is slot status derived?
How does row-level security affect customers and operators?
How do we know billing cannot be typed manually?

One-line answers to keep ready

The database owns correctness.

The UI calls functions and views, but keys, constraints, row locks, triggers and RLS keep the data valid even if a client makes a bad request.

The live demo should prove insert, view and database reflection, then stop.